# PES UNIVERSITY

Department of Computer Science & Engineering

Software Engineering Lab — Lab 7: Fuzz Testing with Hypothesis

| | | | |
|---|---|---|---|
| **Name:** | A S VINCENT | **SRN:** | PES2UG24CS002 |
| **Section:** | A (5th Sem) | **Assigned Repo:** | L6_L7_01_water_utility_billing |

## Step 2: Run Buggy Application (processor.py)

Ran python processor.py on typical valid inputs. The program completes successfully for well-formed strings, but has no validation for missing fields, invalid formats, or null inputs.

```
PS C:\Users\vincent\Downloads\se_lab7\Lab 7> python processor.py
==== Running sanitize_meter_id ====
Enter a meter ID with prefixes (!,@,#,$,%): !@#M-101$%
Sanitized Meter ID: M-101

==== Running parse_consumption_readings ====
Enter consumption readings (e.g. 10,25,30): 10,25,30
Parsed Readings: [10, 25, 30]

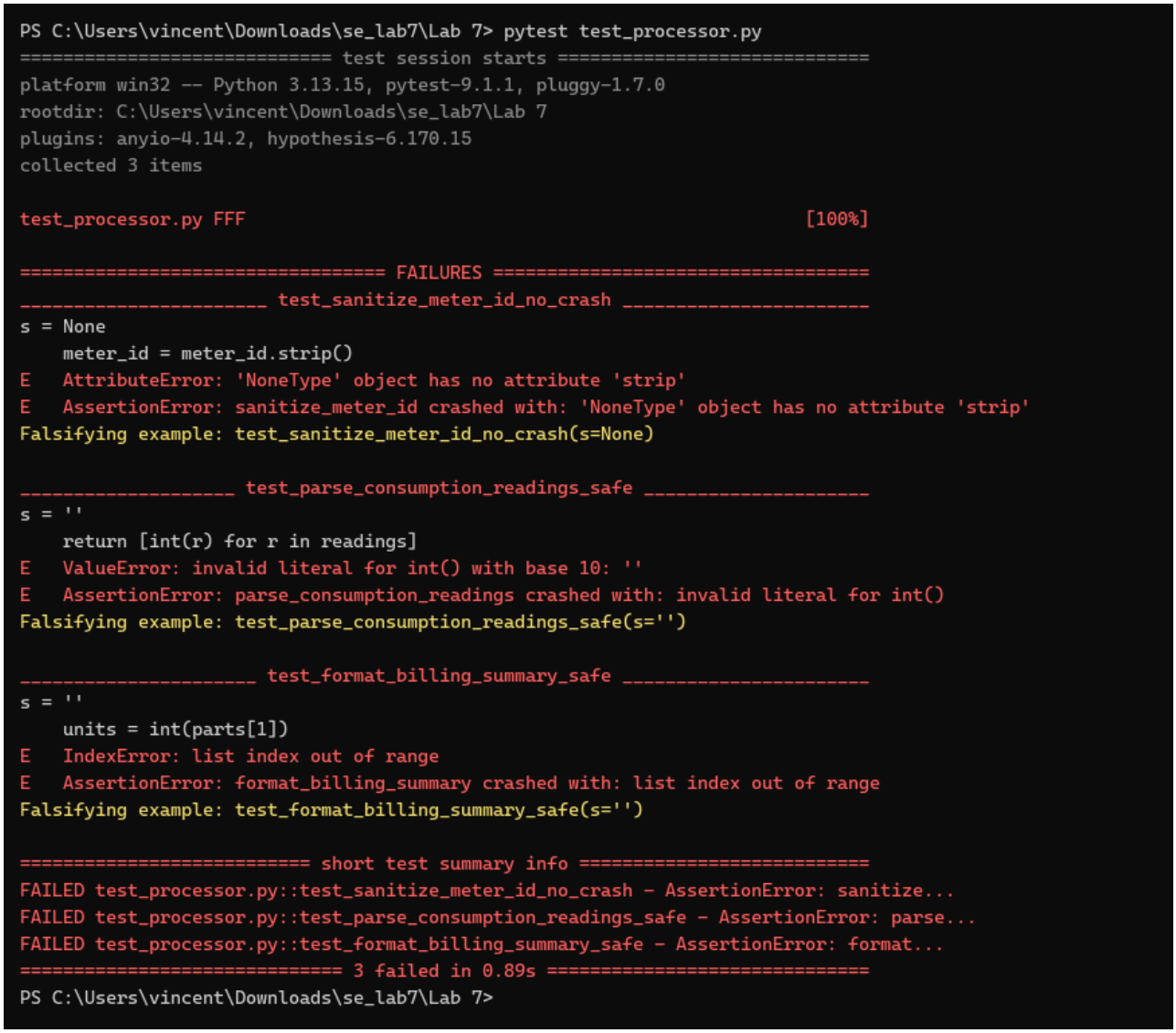
==== Running format_billing_summary ====
Enter account record (Name:Usage:Zone): Alice:50:North
Billing Summary: Customer Alice billed 100 USD in North
PS C:\Users\vincent\Downloads\se_lab7\Lab 7>
```

## Step 4: Run test_processor.py with Hypothesis (Failures)

Completed the test template in test_processor.py using @given(st.text() | st.none()). Running pytest test_processor.py caused all 3 functions to fail:

- **sanitize_meter_id:** Crashed with AttributeError when input is None.
- **parse_consumption_readings:** Crashed with ValueError on empty string and AttributeError on None.
- **format_billing_summary:** Crashed with IndexError and ValueError on records with missing colon delimiters.

## Screenshot: pytest test_processor.py failing on buggy code



```
PS C:\Users\vincent\Downloads\se_lab7\Lab 7> pytest test_processor.py
============================= test session starts =============================
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\vincent\Downloads\se_lab7\Lab 7
plugins: anyio-4.14.2, hypothesis-6.170.15
collected 3 items

test_processor.py FFF                                                    [100%]

================================== FAILURES ===================================
_____________________ test_sanitize_meter_id_no_crash _____________________
s = None
    meter_id = meter_id.strip()
E   AttributeError: 'NoneType' object has no attribute 'strip'
E   AssertionError: sanitize_meter_id crashed with: 'NoneType' object has no attribute 'strip'
Falsifying example: test_sanitize_meter_id_no_crash(s=None)

_________________ test_parse_consumption_readings_safe _________________
s = ''
    return [int(r) for r in readings]
E   ValueError: invalid literal for int() with base 10: ''
E   AssertionError: parse_consumption_readings crashed with: invalid literal for int()
Falsifying example: test_parse_consumption_readings_safe(s='')

_____________________ test_format_billing_summary_safe _____________________
s = ''
    units = int(parts[1])
E   IndexError: list index out of range
E   AssertionError: format_billing_summary crashed with: list index out of range
Falsifying example: test_format_billing_summary_safe(s='')

=========================== short test summary info ===========================
FAILED test_processor.py::test_sanitize_meter_id_no_crash - AssertionError: sanitize...
FAILED test_processor.py::test_parse_consumption_readings_safe - AssertionError: parse...
FAILED test_processor.py::test_format_billing_summary_safe - AssertionError: format...
============================== 3 failed in 0.89s ==============================
PS C:\Users\vincent\Downloads\se_lab7\Lab 7>
```

## Step 5: Run Fixed Application (processor.py)

Fixed processor.py by checking for non-string/null inputs, safely parsing integer lists, and validating delimiter counts. Verified that invalid inputs no longer cause unhandled crashes.

```
PS C:\Users\vincent\Downloads\se_lab7\Lab 7> python processor.py
==== Running sanitize_meter_id ====
Enter a meter ID with prefixes (!,@,#,$,%): !#@WM-204$%
Sanitized Meter ID: WM-204

==== Running parse_consumption_readings ====
Enter consumption readings (e.g. 10,25,30): 10, 25, invalid, 40
Parsed Readings: [10, 25, 40]

==== Running format_billing_summary ====
Enter account record (Name:Usage:Zone): Bob:35:South
Billing Summary: Customer Bob billed 70 USD in South
PS C:\Users\vincent\Downloads\se_lab7\Lab 7>
```

## Step 6: Re-run test_processor.py on Fixed Code (All Passing)

Ran pytest test_processor.py -v on the fixed module. All three property-based fuzz tests passed.

```
PS C:\Users\vincent\Downloads\se_lab7\Lab 7> pytest test_processor.py -v
============================= test session starts =============================
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\vincent\Downloads\se_lab7\Lab 7
plugins: anyio-4.14.2, hypothesis-6.170.15
collecting ... collected 3 items

test_processor.py::test_sanitize_meter_id_no_crash PASSED                [ 33%]
test_processor.py::test_parse_consumption_readings_safe PASSED           [ 66%]
test_processor.py::test_format_billing_summary_safe PASSED               [100%]

============================== 3 passed in 0.57s ==============================
PS C:\Users\vincent\Downloads\se_lab7\Lab 7>
```

## Reflection Questions

**1. What types of bugs did the fuzzer expose that normal tests missed?**

The fuzzer exposed null (None) pointer errors, empty string crashes during int conversion, and missing delimiter index errors that standard unit tests with valid sample inputs did not catch.

**2. How does Hypothesis shrinking help?**

When a failure is found with random data, Hypothesis reduces the input to the simplest possible value (like None or ""), making it clear what edge case caused the failure.

**3. Why is fuzz testing useful for input validation?**

It automatically tests random and unexpected combinations of inputs without manual effort, helping ensure that parsers and user-facing functions handle invalid data safely.